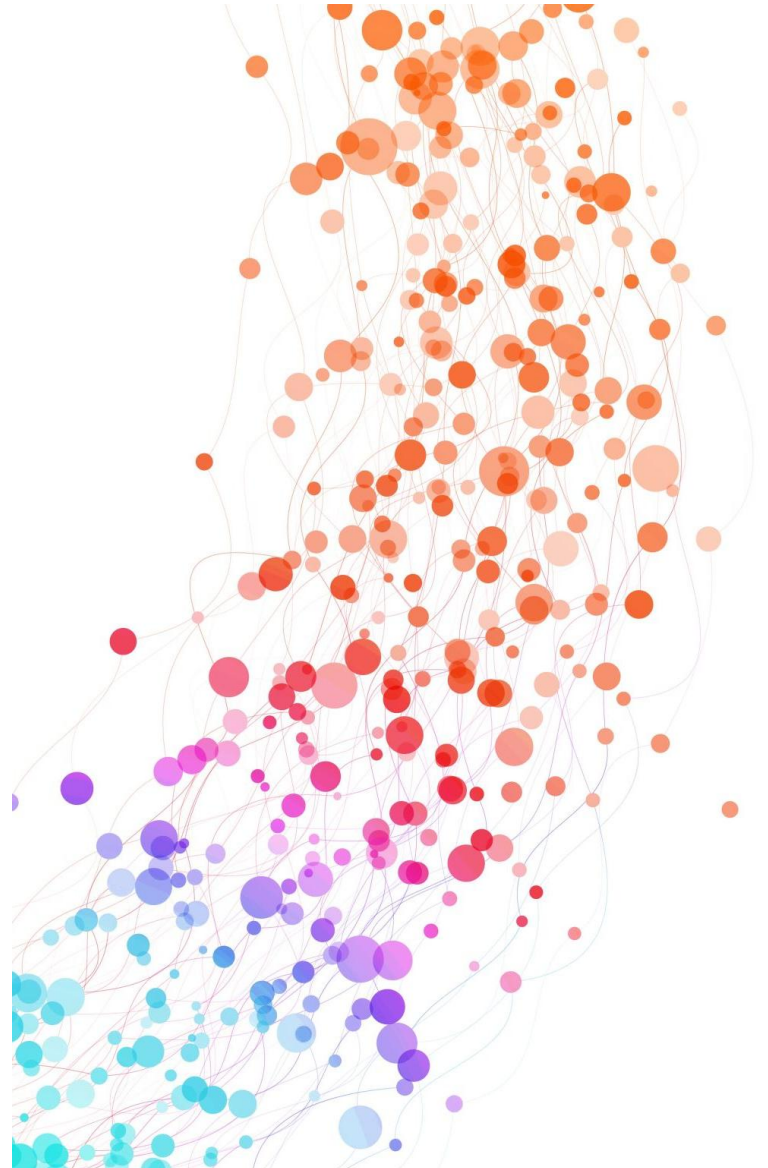


ソート解説

3年情報



先生：いきなりたくさんのデータを並び替える前に、2つの数値を入れかえるプログラムを考えてみましょう。

花子：A00とA01の利用者の満足度の数値を入れかえてみましょう。Tensu[0]が20、Tensu[1]が17のとき、Tensu[0]の値をkariに、Tensu[1]の値をTensu[0]に、kariの値をTensu[1]にそれぞれ代入すればよいので、図1のようなプログラムになりますね。

```
(1) Tensu = [20, 17]
(2) kari = Tensu[ ア ]
(3) Tensu[0] = Tensu[1]
(4) Tensu[ イ ] = kari
```

図1 2個のデータを入れかえるプログラム

ア ・ イ の解答群

① 0

② 1

③ 2

答え ア 0

先生：いきなりたくさんのデータを並び替える前に、2つの数値を入れかえるプログラムを考えてみましょう。

花子：A00とA01の利用者の満足度の数値を入れかえてみましょう。Tensu[0]が20、Tensu[1]が17のとき、Tensu[0]の値をkariに、Tensu[1]の値をTensu[0]に、kariの値をTensu[1]にそれぞれ代入すればよいので、図1のようなプログラムになりますね。

```
(1) Tensu = [20, 17]
(2) kari = Tensu[ ア ]
(3) Tensu[0] = Tensu[1]
(4) Tensu[ イ ] = kari
```

図1 2個のデータを入れかえるプログラム

ア ・ イ の解答群

① 0

② 1

③ 2

答え イ ①

先生：図1は正しく動作します。では次に，A00 から A19 の利用者の満足度の数値の中から一番小さい数値を先頭に移動させてみましょう。j を 1 から まで動かして，Tensu[j] と Tensu[0] の大小を比較して，Tensu[j] の方が小さいときに 2 つの数値を入れかえます。

花子：次のようなプログラムになりますね。結果も表示させてみます。

```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
            17, 20, 1, 12, 15, 17]
(2) j を 1 から  まで 1 ずつ増やしながら繰り返す：
(3)     もし Tensu[j] < Tensu[0] ならば：
(4)         kari = Tensu[  ]
(5)         Tensu[0] = Tensu[j]
(6)         Tensu[  ] = kari
(7) 表示する (Tensu)
```

図2 20個のデータの最小値を先頭にするプログラム

- (3)でtensu[j]と tensu[0]を比較している
- jを0から始めると tensu[0]とtensu[0]の比較になり意味がない
- そのためjを1=17から始め最後の17になるところまで繰り返す

答え ③19

先生：図1は正しく動作します。では次に、A00からA19の利用者の満足度の数値の中から一番小さい数値を先頭に移動させてみましょう。jを1から まで動かして、Tensu[j]とTensu[0]の大小を比較して、Tensu[j]の方が小さいときに2つの数値を入れかえます。

花子：次のようなプログラムになりますね。結果も表示させてみます。

```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
            17, 20, 1, 12, 15, 17]
(2) jを1から  まで1ずつ増やしながら繰り返す:
(3)     もし Tensu[j] < Tensu[0]ならば:
(4)         kari = Tensu[  ]
(5)         Tensu[0] = Tensu[j]
(6)         Tensu[  ] = kari
(7) 表示する (Tensu)
```

図2 20個のデータの最小値を先頭にするプログラム

- (5)行目でtensu[j]=17の値を tensu[0]に入れている
これをするとtensu[0]
が上書きされて消えてしまう
- tensu[0]が消えないように
一旦kariに保存しておく

答え 0

先生：図1は正しく動作します。では次に、A00からA19の利用者の満足度の数値の中から一番小さい数値を先頭に移動させてみましょう。jを1から まで動かして、Tensu[j]とTensu[0]の大小を比較して、Tensu[j]の方が小さいときに2つの数値を入れかえます。

花子：次のようなプログラムになりますね。結果も表示させてみます。

```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
            17, 20, 1, 12, 15, 17]
(2) jを1から  まで1ずつ増やしながら繰り返す:
(3)     もし Tensu[j] < Tensu[0]ならば:
(4)         kari = Tensu[  ]
(5)         Tensu[0] = Tensu[j]
(6)         Tensu[  ] = kari
(7) 表示する (Tensu)
```

図2 20個のデータの最小値を先頭にするプログラム

- いま kari には tensu[0] の値である 20 が入っている
- tensu[j] = 17 を tensu[0] に代入した後、元々 tensu[0] に入っていた 20 を tensu[j] に移動させる

答え ④ j

```
(1) Tensu = [1, 20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 15, 12, 14,
             17, 20, 1, 12, 15, 17]
(2) j を 2 から  まで 1 ずつ増やしながら繰り返す:
(3)     もし Tensu[j] < Tensu[  ] ならば:
(4)         kari = Tensu[  ]
(5)         Tensu[  ] = Tensu[j]
(6)         Tensu[  ] = kari
(7) 表示する (Tensu)
```

図3 2番目以降のデータの中から最小の数値を先頭から2番目にするプログラム

キ ~ ケ の解答群		
① 0	② 1	③ 2
④ j - 1	⑤ j	⑥ j + 1

- 2番目に一番小さい数字が来るようにする
- 1番目の1は無視
- まずTensu[j]=17 と20を比較する
- 20はTensu[1]に入っている

答え ①1

```

(1) Tensu = [1, 20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 15, 12, 14,
             17, 20, 1, 12, 15, 17]
(2) j を 2 から  まで 1 ずつ増やしながら繰り返す:
(3)     もし Tensu[j] < Tensu[  ] ならば:
(4)         kari = Tensu[  ]
(5)         Tensu[  ] = Tensu[j]
(6)         Tensu[  ] = kari
(7) 表示する (Tensu)
    
```

図3 2番目以降のデータの中から最小の数値を先頭から2番目にするプログラム

キ ~ ケ の解答群		
① 0	② 1	③ 2
④ j - 1	⑤ j	⑥ j + 1

- Tensu[j]=17とTensu[l]=20を比較すると17の方が小さいので下に行ける
- l番目のlは無視
- (5)でtensu[j]=17をTensu[l]に入るので、その前にTensu[l]をkariに保存させておく

答え ① l


```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
             17, 20, 1, 12, 15, 17]
(2) i を 0 から  コ  まで 1 ずつ増やしながら繰り返す:
(3)     j を i + 1 から  サ  まで 1 ずつ増やしながら繰り返す:
(4)         もし Tensu[j] < Tensu[  シ  ] ならば:
(5)             kari = Tensu[  ス  ]
(6)             Tensu[  シ  ] = Tensu[j]
(7)             Tensu[  セ  ] = kari
(8) 表示する (Tensu)
```

図4 表1のデータを昇順に表示するプログラム

- Tensuの配列は0から19までである
- 比較してるのはTensu[i]とTensu[j]
- jはi+1なので1番目からスタート
- iは0からスタート
- 最初はTensu[i]が20、Tensu[j]が17
- そのまま比較を続けると最後は15と17の比較になりiは18までで、jは19までになる

答え コ 0 18 サ ①19

```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
             17, 20, 1, 12, 15, 17]
(2) i を 0 から  コ  まで 1 ずつ増やしながら繰り返す:
(3)     j を i + 1 から  サ  まで 1 ずつ増やしながら繰り返す:
(4)         もし Tensu[j] < Tensu[  シ  ] ならば:
(5)             kari = Tensu[  ス  ]
(6)             Tensu[  シ  ] = Tensu[j]
(7)             Tensu[  セ  ] = kari
(8) 表示する (Tensu)
```

図4 表1のデータを昇順に表示するプログラム

- 比較してるのはTensu[i]とTensu[j]
- jはi+1なので1番目からスタート
- iは0からスタート
- 最初はTensu[i]が20、Tensu[j] が17

答え ③ i

```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
            17, 20, 1, 12, 15, 17]
(2) i を 0 から  まで 1 ずつ増やしながら繰り返す:
(3)   j を i + 1 から  まで 1 ずつ増やしながら繰り返す:
(4)       もし Tensu[j] < Tensu[  ] ならば:
(5)           kari = Tensu[  ]
(6)           Tensu[  ] = Tensu[j]
(7)           Tensu[  ] = kari
(8) 表示する (Tensu)
```

図4 表1のデータを昇順に表示するプログラム

- 大きい順に並べることを考える
最初はTensu[i]が20、Tensu[j]が17で比較
- Tensu[i]の方が大きいので入れ替えが起きる
- Tensu[j]をTensu[i]に代入するのでTensu[i]を一旦kariに保存させる

答え ③ i

```
(1) Tensu = [20, 17, 11, 15, 18, 12, 19, 20, 19, 2, 1, 15, 12, 14,
            17, 20, 1, 12, 15, 17]
(2) i を 0 から  コ  まで 1 ずつ増やしながら繰り返す:
(3)     j を i + 1 から  サ  まで 1 ずつ増やしながら繰り返す:
(4)         もし Tensu[j] < Tensu[  シ  ] ならば:
(5)             kari = Tensu[  ス  ]
(6)             Tensu[  シ  ] = Tensu[j]
(7)             Tensu[  セ  ] = kari
(8) 表示する (Tensu)
```

図4 表1のデータを昇順に表示するプログラム

- Tensu[j]をTensu[i]に代入するのでTensu[i]を一旦kariに保存させる
- 保存したTensu[i]を最後にTensu[j]に入れる

答え

④ j

要素数＝配列のデータ数

1	2	3	4	5	6	7	8	9	10
48	21	52	48	17	29	10	34	48	34

答え ③10


```
(07)  iを0から  ウ  まで1ずつ増やしながら繰り返す :  
(08)  |      もしData[i] == ataiならば :  
(09)  |          kekka =  エ
```

- ・ もし探しているデータが見つければkekkaに何かが代入される
- ・ 探しているものがなければkekkaは初期値(0)から変わらない

答え 0

0	1	2	3	4	5	6	7	8	9
48	21	52	48	17	29	10	34	48	34

Data = [48, 21, 52, 48, 17, 29, 10, 34, 48, 34]

- ・ 配列は0から始まる
- ・ 今回は9回まで繰り返す
- ・ kazuには10が入っているので9になるようにする

答え ②kazu-1

表示する（”見つかりました：”，kekka, ”番目”）

- ・ 見つかった場合iがその時の数字である
- ・ そのたまiを代入したらいいが、kekka=0で見つかりませんでしたと表示されてしまうので添字の0が1番目になるようにする

答え ④ $i+1$

```
(07)  iを0から  ウ  まで1ずつ増やしながら繰り返す :  
(08)  |      もしData[i] == ataiならば :  
(09)  |          kekka =  エ
```

- ・ もし探しているデータが見つければkekkaに何かが代入される
- ・ 探しているものがなければkekkaは初期値(0)から変わらない

答え kekka==0

```
(06)  iを0から  ウ  まで1ずつ増やしながら繰り返す :  
(07)  |      もしData[i] == ataiならば :  
(08)  |      kekka =  エ
```

- ・ 探している値が見つかったときに繰り返しを抜きたい

答え ③ (08) 行目直下

```
(06)  iを0から  ウ  まで1ずつ増やしながら繰り返す :  
(07)  |      もしData[i] == ataiならば :  
(08)  |          kekka =  エ
```

- ・ 探している値が見つかったときに繰り返しを抜きたい

答え ③ (08) 行目直下

0 1 2 3 4 5 6 7 8 9

Data = [48, 21, 52, 48, 17, 29, 10, 34, 48, 21]

- ・ 上のように配列は0から順番に前から見ていく
- ・ 同じ数があった場合は最後に来ているものが優先させる

答え ②最後

0 1 2 3 4 5 6 7 8 9

Data = [48, 21, 52, 48, 17, 29, 10, 34, 48, 21]

- ・ 配列の中に探索値が1つだけの場合kosuは+1されて1になる
- ・ 上のように配列の中に複数個21があった場合kosuが2になり1より大きくなる

答え ②kosu>1